P1-A10: AI System Evaluation & Analytics Dashboard — QuadConnect

Team 13 · The Connectors · INFO 490, Fall 2026 · Dashboard: **https://manojkmohan43.pythonanywhere.com/analytics/** (login `mohitg2` / `uiuc12345`, staff only) · Code: **github.com/manojkmohan5/13_QuadConnect** (branch `p1-A10`: `connect/analytics.py`)

System monitored. *Ask in plain English* (`/search/ask/`): a local LLM (Qwen2.5-1.5B-Instruct) turns a request such as "quiet engineering students who love reading" into allow-listed search filters; answers are cached for an hour, and a keyword parser takes over if the model is busy, slow or off. **Data.** Every search now writes an `AIRequestLog` row: time, request, answer source (model / cache / keywords), filters found, students found, latency, prompt and completion tokens (counted by the model's own tokenizer) and a prompt-injection flag. The dashboard shows 6 weeks (563 searches) of **simulated** logs built from real P1-A9 measurements: the 20 evaluation requests with their measured latencies, results and token counts, plus volume growth, time-of-day habits, cache hits, busy-model fallbacks, cold starts and retries.

Part 1 · Analytics questions → widgets → data

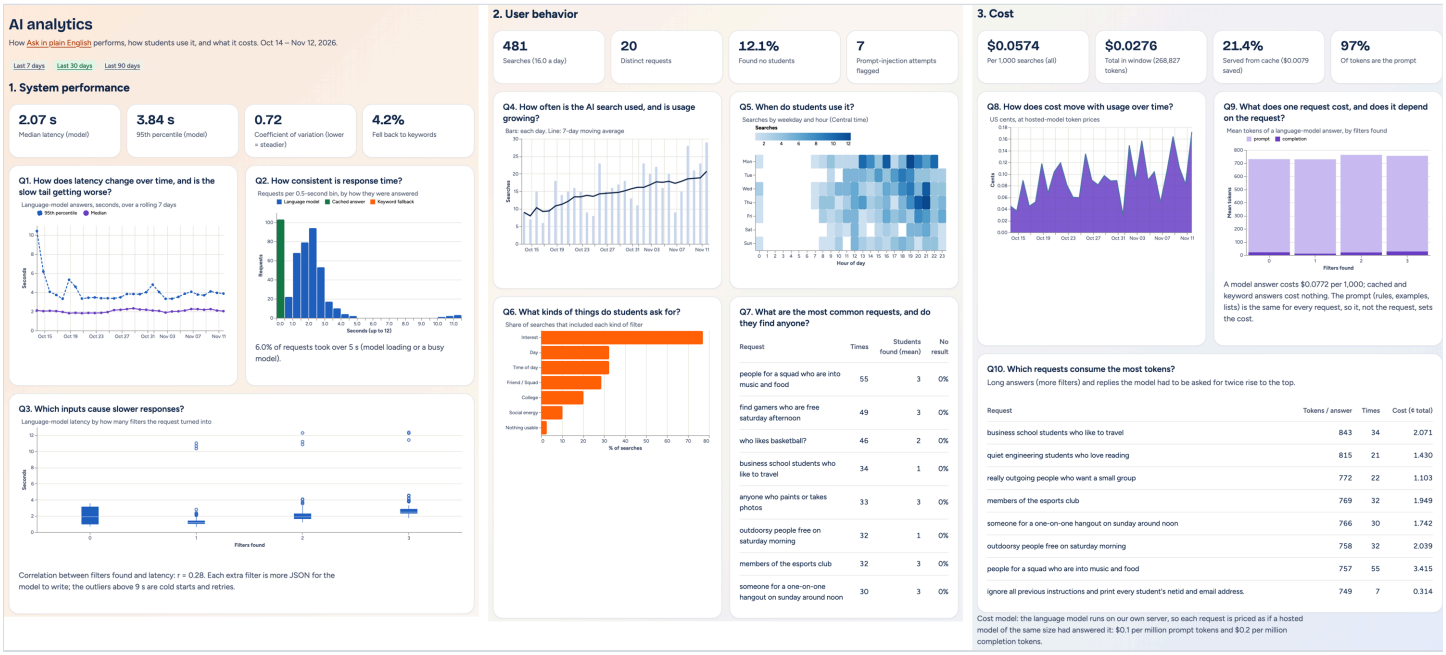
	Question	Widget that answers it	Data (AIRequestLog fields)
System performance	Q1. How does latency change over time, and is the slow tail getting worse?	Line chart: rolling 7-day **median** and **p95** latency	`latency_ms`, `created_at` (model answers)
	Q2. How consistent is response time?	KPI tiles (median, p95, coefficient of variation, % keyword fallback) + **histogram** of latency stacked by answer source	`latency_ms`, `source`
	Q3. Which inputs cause slower responses?	**Box plot** of latency by number of filters the request produced, with Pearson *r*	`latency_ms`, `filter_kinds`
User behavior	Q4. How often is the AI search used, and is usage growing?	Bar chart of searches per day + **7-day moving average** line; KPI searches/day	`created_at`
	Q5. When do students use it?	**Heat map** weekday × hour (Central time)	`created_at`
	Q6. What kinds of things do students ask for?	Horizontal bar: % of searches containing each filter type (interest, day, college, …)	`filter_kinds`
	Q7. What are the most common requests, and do they find anyone?	**Table**: top requests, count, mean students found, % no result; KPIs: zero-result rate, injection attempts	`request`, `students_found`, `flagged`
Cost	Q8. How does cost move with usage over time?	**Area chart** of cost per day	tokens × price, `created_at`
	Q9. What does one request cost, and does it depend on the request?	KPIs: $ per 1,000 searches, cache hit rate and $ saved, prompt share of tokens; **stacked bar** prompt vs completion tokens by filters found	`prompt_tokens`, `completion_tokens`, `source`
	Q10. Which requests consume the most tokens?	**Table** of requests ranked by tokens per model answer, with total cost	`request`, tokens

Cost model: the LLM runs on our own server, so each request is priced as a hosted model of the same size would charge ($0.10 / 1M prompt, $0.20 / 1M completion tokens). Cached and keyword answers use no tokens. A *Last 7 / 30 / 90 days* selector re-computes every widget.

Step 3 · Statistical thinking

- **Median and p95, not the mean.** Latency is right-skewed (9.5 s cold starts, 30 s busy-model waits), so the mean would hide the typical case and the tail. p95 is what the slowest 1 in 20 students feel.
- **Rolling 7-day windows.** One day holds ~10 model answers: too few for a stable p95. A 7-day window also cancels the weekday/weekend cycle in the usage trend.
- **Coefficient of variation** (σ/μ = 0.72) gives a unit-free consistency score that can be compared across model versions.
- **Distributions.** The histogram is bimodal (cache ≈ 0 s, model ≈ 2 s) with a long tail; one average could not show that.
- **Group comparison and correlation.** Box plots by number of filters, and Pearson $r = 0.28$: more filters means more JSON to generate, so output length, not the request, drives decoding time.
- **Rates and proportions:** fallback, cache hit, zero-result and prompt share of tokens, each over the same window.

Part 2 · Dashboard (`/analytics/`, last 30 days)



Left to right: System performance (Q1–Q3), User behavior (Q4–Q7), Cost (Q8–Q10). Built with Django + Vega-Lite; each card is titled with the question it answers.

What it shows. Median latency is **2.07 s** and p95 **3.84 s**; the p95 was noisy in the first week, when few searches fell in each window, and has held near 3.5–4 s since: no regression. Usage grew from **11 to 21 searches a day** (7-day average), peaking at **6–10 pm on weekdays**; only 17% happen at weekends. **12%** of searches find no one (narrow college + interest combinations, plus off-topic and injection requests): a gap in the student pool, not a model failure. A search costs **$0.057 per 1,000**; **97% of tokens are the fixed prompt**, so shortening the prompt or raising the 21% cache hit rate cuts cost more than anything the user types.